Personalised Python Tutor Using Student Knowledge Modelling

Mid-Submission Report

Priyanshu Panchasara (2026701004) Jenil Kalsariya (2026701029)
Iska Puneeth Abhishek (2026701051) Vidhi Jain (2026701024)

GitHub: `https://github.com/vidhijain0204/LMA_Group_Project`

Abstract

Beginners learn Python at different speeds, but a general AI chatbot treats every learner the same and often just gives the final answer. In this project we are building a personalised Python tutor that adapts to each student. The system uses three main ideas: Agentic AI, Retrieval-Augmented Generation (RAG), and Reasoning. A central Orchestrator decides which agent should act next, and all agents run on a small language model (SLM). A diagnosis step checks what a new student already knows, a Tutor Agent explains concepts and gives hints, and an Assessment Agent checks the student's answers. The Tutor uses RAG, so its answers come from real course material instead of guesses, and it reasons about the student's level to choose a simpler or a more advanced explanation. For this, we collected Python learning material from public sources and also built a question bank of 455 MCQs covering 13 topics and 74 subconcepts. We tested four ways of finding the right course material and chose the best one for the Tutor. Next, we will build the Error Analysis Agent and test whether personalisation helps students learn better.

Keywords: Agentic AI, Retrieval-Augmented Generation, Reasoning, Intelligent Tutoring System, Python Education

1 Introduction

Learning to program is hard for beginners. Every student starts with different knowledge and makes different mistakes, so a fixed set of lessons does not suit everyone. General AI chatbots answer quickly, but they often give the full solution right away, and they do not remember what a student knows.

Our goal is a Python tutor that behaves more like a good teacher: it finds out what the student already knows, explains at the right level, checks understanding, and gives hints instead of only answers. We use three ideas:

- **Agentic AI:** several small agents each do one job, and a central Orchestrator decides who acts next.
- **RAG:** the Tutor first looks up the right material from a Python course dataset and then answers using it.
- **Reasoning:** the system thinks about the student's level and situation before deciding what to do.

Progress so far:

Table 1: Progress summary

Part	Status	Details
Datasets	Done	Course corpus, 455-question MCQ bank, 26 retrieval test queries
Orchestrator, Diagnosis, Assessment	Done	Working in one LangGraph pipeline with the SKB
Tutor Agent with RAG	Done	BM25 retrieval chosen from four methods
Error Analysis Agent	To do	Planned in days 1–5 of the final plan

2 Data Collection

2.1 Course Corpus

We collected Python learning material from public websites and textbooks, including W3Schools [1]. The steps were: extract the text, clean it, split it into small chunks, tag each chunk with a topic, and save useful details such as difficulty and source. Code examples and the text around them were kept because they help students. The result is `data/common/corpus.jsonl`, one chunk per line.

2.2 MCQ Bank

We wrote 455 Python multiple-choice questions, balanced across 13 topics (35 per topic) and covering all 74 subconcepts of our topic list. The questions were written by us and not copied from the course material. Each question has four options, the correct option, a difficulty label, its main topic with related or prerequisite topics, and an explanation. Eight question types are included: output prediction, debugging, code completion, concept understanding, best practice, application, error identification, and edge cases. Difficulty uses six starting labels from 0 (very easy) to 1 (very hard); these are our own labels, not tested scores. The bank is stored in `python_tutor_mcqs.json`, and it is for development and testing, not a standard exam.

2.3 Retrieval Test Queries

To compare retrieval methods we prepared a fixed set of 26 queries (`retrieval_eval.csv`); 22 have the correct chunks marked and 4 do not. These are not real student chats, only a fixed test set.

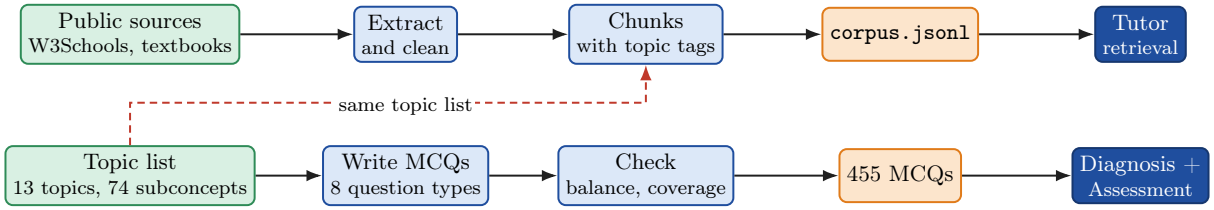


Figure 1: How the course corpus and the MCQ bank were prepared.

Table 2: Summary of datasets

Dataset	Contents	Used for
Course corpus	Chunks of Python lessons with topic, type, difficulty, source	RAG for the Tutor
MCQ bank	455 questions, 13 topics, 74 subconcepts	Diagnosis and Assessment
Test queries	26 queries (22 with correct chunks marked)	Comparing retrieval methods

3 Agentic System

The tutor is built as small agents that each do one job, with a central Orchestrator choosing who acts. All of them share one student record, the Student Knowledge Base (SKB). We kept the SKB small on purpose: we store a detail only if it changes a decision, such as which topic to teach, which material to fetch, or which question to ask. For each topic it keeps the student's knowledge level, how much evidence supports it, recent results, and common mistakes. Topics use the same list as the MCQ bank and the course corpus, and correct answers are never shown to the Tutor.

3.1 Common Flow

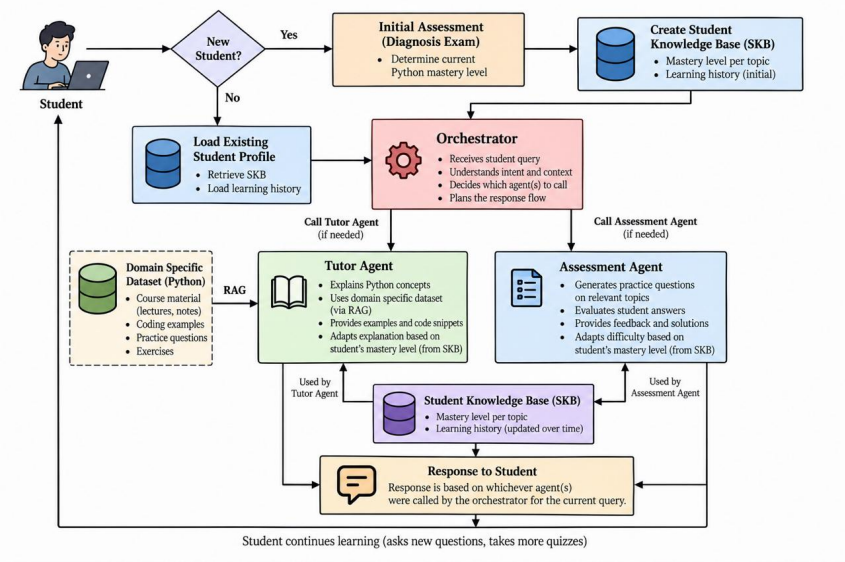


Figure 2: Complete flow of the agentic system.

Figure 2 shows the flow:

1. The student starts a session and the system checks whether the student is new.
2. A new student takes the Initial Assessment (Diagnosis Exam), which finds the current Python mastery level, and the SKB is created. An existing student has the SKB and learning history loaded.
3. The Orchestrator receives the student's query, understands the intent and context, and decides which agent to call.
4. The Tutor Agent is called for concepts and the Assessment Agent is called for practice, as needed. Both read the student's mastery level from the SKB.
5. The response to the student is based on whichever agent was called, and the student continues learning with new questions or more quizzes.

The Orchestrator calls only the part that the situation needs, not every part for every message.

Table 3: Who is called when

Situation	Called	Why
New student, no record	Diagnosis	Find the starting level
Concept question or hint request	Tutor Agent	Explain using course material
Student submits an answer	Assessment Agent	Check the answer
Student wants the next exercise	Assessment Agent	Pick a suitable question

3.2 Orchestrator

The Orchestrator is the entry point of a LangGraph state graph. All agents share one state (student record, messages, topic focus, quiz results, score history), and the Orchestrator reads it to decide which agent acts next. It first applies a simple rule: if the student has no SKB record, the student goes straight to diagnosis, because a missing record is a fact from the database and not something to guess. Otherwise the SLM reads the message and returns the next agent, the topic focus, an instruction and a short reason as JSON. Since small models can return messy output, we clean and check the JSON, map labels like "quiz" to real agents, and fall back to the Tutor if parsing fails. Existing students are never sent back to diagnosis.

3.3 Diagnosis for New Students

A new student has no record, so the system does not know their level. The Initial Assessment (Diagnosis Exam) is a fixed test of basic Python from our MCQ bank. It is a test, not an autonomous agent, and each student can take it only once. The student sees the questions without the correct answers. Each answer is checked against the stored option, and accuracy is calculated per topic and subconcept. Topics scoring above 0.7 are marked strengths and the others weaknesses, and every wrong answer adds to the count of the common mistakes linked to that question. The result is saved as the student's SKB before any teaching starts, so the first explanation is not too easy or too hard.

3.4 Tutor Agent

The Tutor Agent explains Python concepts and gives hints, examples and code snippets, using RAG over the domain-specific course data instead of guessing. For each question, its prompt includes the focus topic, the student's mastery score for that topic, their active misconceptions and the Orchestrator's instruction. A weak student gets a simple real-life analogy, a step-by-step explanation and a very basic example. A developing student gets a standard explanation, and a strong student skips the basics and gets an edge case or a common pitfall. The reply is cleaned so that prompts and thinking text are never shown, and the SKB can be switched off to compare the Tutor with and without student information. How it finds the course material is described in Section 4.

3.5 Assessment Agent

The Assessment Agent checks what the student has learned. It finds the topic, reads the student's score, and sets an easy, medium or hard target difficulty. It then picks the 10 questions from the MCQ bank closest to that difficulty. Each answer is checked against the stored correct option, so the score does not depend on the SLM's opinion, and the SLM only writes a short, encouraging introduction. The accuracy, the graded answers, the wrongly answered questions and the score history are saved, and they will be the input for the Error Analysis Agent.

3.6 Implementation Notes

- **Pause and resume:** the diagnosis test and the quiz pause the graph, wait for the student's answers, and then continue with the same session.
- **Problems fixed:** the Tutor was running alongside every diagnosis and quiz request, so we kept only the Orchestrator's routing. A quiz drew a new random set of questions when the student submitted, so we now grade the questions the student actually answered. Model outputs leaked prompt and thinking text, so we added output cleaning.

4 RAG Implementation

The Tutor does not answer from the SLM's memory alone. It first finds the best course chunks and gives them to the SLM together with the student's level.

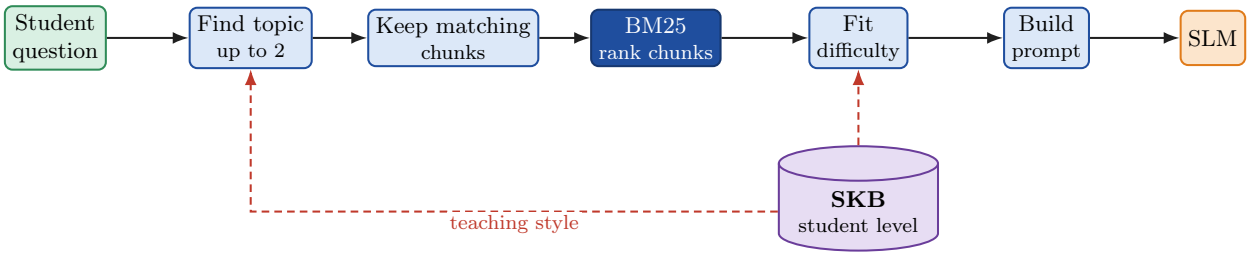


Figure 3: How the Tutor finds course material and answers.

Retrieval methods. The retriever reads the corpus, lowercases the text, and keeps topic, difficulty and source for each chunk. We compared four methods: TF–IDF (a basic keyword score), BM25 (an improved keyword score that handles repeated words and chunk length [2]), dense retrieval (compares meaning using `all-MiniLM-L6-v2` [3]), and Hybrid, which combines BM25 and dense

rankings by Reciprocal Rank Fusion [4]: $\mathrm{RRF}(d) = \sum_r 1/(60 + \mathrm{rank}_r(d))$.

Topic and level. The Tutor finds up to two topics for the question and searches only those chunks (all chunks if none match). The student's knowledge level then picks a target difficulty: below 0.40 gives difficulty 1 with basic explanations, 0.40 to 0.70 gives difficulty 2 to fill gaps, and 0.70 or more gives difficulty 3 with direct or advanced material. The ranking is adjusted by $S_{\text{final}} = S_{\text{relevance}} \times (1 + 0.10F)$, where F says how well a chunk's difficulty fits. The boost is at most 10%, so relevance stays the main signal. The Streamlit app creates the retriever once and reuses it.

Results. We ran `scripts/compare_retrievers.py` on the 26 test queries, looking at the top 5 results. Recall@5 is the share of correct chunks found, MRR shows how early the first correct chunk appears, and latency is the time per query.

Table 4: Retrieval comparison (top 5 results)

Method	Recall@5	MRR	Mean latency (ms)
TF–IDF	0.7379	0.7083	17.314
BM25	**0.8356**	0.7629	**2.392**
Dense	0.4955	0.6818	18.788
Hybrid (RRF)	0.7061	**0.7917**	25.692

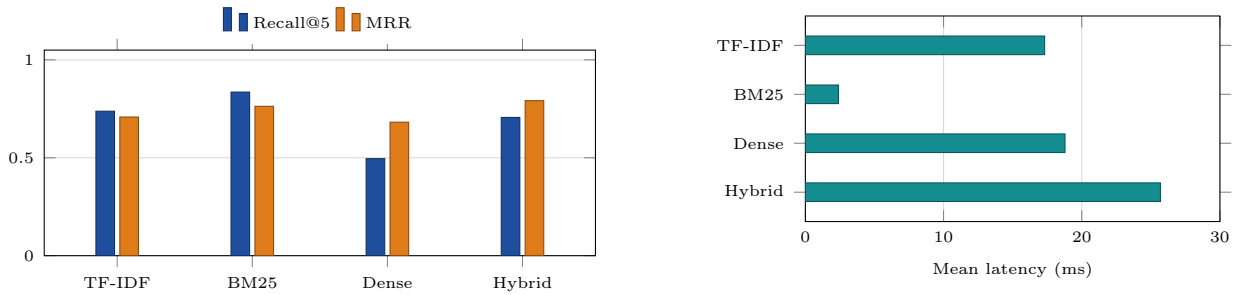


Figure 4: Quality (left) and speed (right) of the four methods.

BM25 found the most correct chunks and was the fastest. Hybrid had the best MRR but was the slowest. TF–IDF beat dense retrieval on both Recall@5 and MRR. Starting the dense model took about 10,808 ms, but this happens only once. We therefore use BM25 in the Tutor, and Hybrid is an option if the rank of the first correct result matters more than speed. This choice is for our current data and queries only. The retriever unit tests passed (10 tests in 0.16 s) but they do not cover the whole app or the SLM answers. The test set is small, and these results measure search quality and speed only, not how good the explanations are.

5 Experiments, Challenges and Plan for Final Submission

5.1 Experiments: Baselines, Ablations and Results

Baselines. For retrieval, TF–IDF, dense and Hybrid search are compared with BM25 (Table 4). For personalisation, the baseline is the RAG Tutor without the KB. **Ablations.** We switch the KB on and off while everything else stays the same (course data, BM25, SLM, questions, prompt), so only whether the Tutor sees the student's level changes. The planned ablations remove the Error Analysis Agent and change which topic is used for the error reason.

Table 5: Experiment status and results

Experiment	Type	Status	Result
Retrieval: TF–IDF, BM25, Dense, Hybrid	Baseline	Done	BM25 best: Recall@5 0.8356, 2.4 ms
RAG Tutor without KB vs with KB	Ablation	Done	Tutor responses compared
RAG with the updated KB: how the Tutor picks related topics and material by difficulty	Ablation	Planned	–
Complete pipeline with the Error Analysis Agent	Full system	Planned	–
Error reason: (a) without KB, (b) with KB, lowest-score topic, (c) with KB, hypothesis-based topic	Ablation	Planned	–

The last experiment aims to find which setting best finds the actual reason behind a student's error. Results from scripted students will be marked as simulated.

5.2 Challenges and Mitigation

Area	Challenge	Mitigation
Data collection	Cleaning, chunking and topic-tagging the course material.	Maintain a shared topic taxonomy across the corpus and MCQ bank; retain difficulty and source metadata for each chunk.
Schema design	Deciding what student information the KB should store.	Store only information that influences tutoring decisions, keeping the KB structured and lightweight.
Agentic calls	Routing requests to the appropriate component and handling inconsistent small-model outputs.	Use rule-based routing where possible, validate structured JSON outputs, and retain the Tutor as a fallback.
RAG	Retrieving relevant course material at an appropriate difficulty level.	Compare four retrieval methods and use BM25 as the default, with a bounded mastery-aware difficulty adjustment.
Corpus coverage	BM25 fails to retrieve relevant chunks for some queries because the existing domain corpus does not cover all required concepts.	Expand the corpus using additional Python textbooks and reliable learning resources, then re-chunk, re-index and re-evaluate retrieval.
Hardware and latency	Dense and hybrid retrieval introduce higher computational overhead and query latency compared with BM25.	Use BM25 as the default retrieval method, while retaining dense and hybrid retrieval for experimentation when computational resources permit.

5.3 Plan for the Final Submission (15 days)

Days	Work
1–5	Build the Error Analysis Agent
6–9	Evaluate the full pipeline
10–12	Improve the pipeline and handle edge cases
13–15	Final check, demo, updated GitHub with all code, and submission

The GitHub repository will be updated with all the code and will contain the data scripts, datasets, retriever, agents, SKB code, experiment scripts, tests, example settings and reproduction steps. API keys and hidden answer fields will not be committed.

Conclusion. We have built the Orchestrator, the diagnosis step, the Tutor Agent with RAG and

the Assessment Agent, with a course corpus and a 455-question MCQ bank. BM25 gave the best recall and speed, so the Tutor uses it. Next we will build the Error Analysis Agent and test whether student information makes the tutor better.

References

[1] W3Schools, "Python Tutorial." Available: `https://www.w3schools.com/python/`. Accessed: Oct. 30, 2026.

[2] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992.

[4] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. 32nd ACM SIGIR, 2009, pp. 758–759.